UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 5: CUDA Memory Model

Volodymyr Kindratenko

2/3/2026

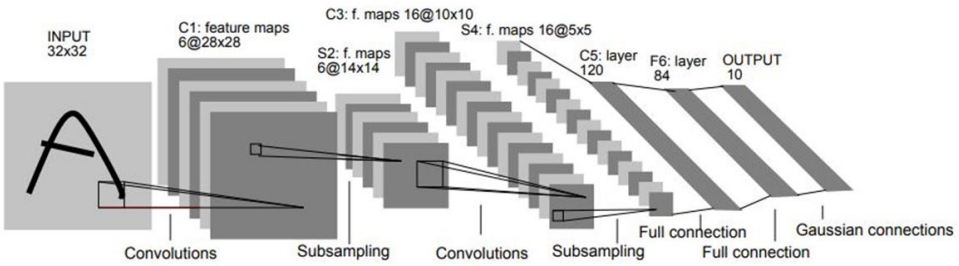
Course Reminders

- Lab 2 is due this Friday
  - Requires Labs 0 and 1 to be completed first
  - Included in the final grade!
- Post your questions on Campuswire, the course staff only replies to questions posted on Campuswire
  - But before posting a question, make sure you try to find the answer yourself first, <u>including asking the course chatbot</u>
- Check office hours schedule, we have in-person and on-line

About the Class Project

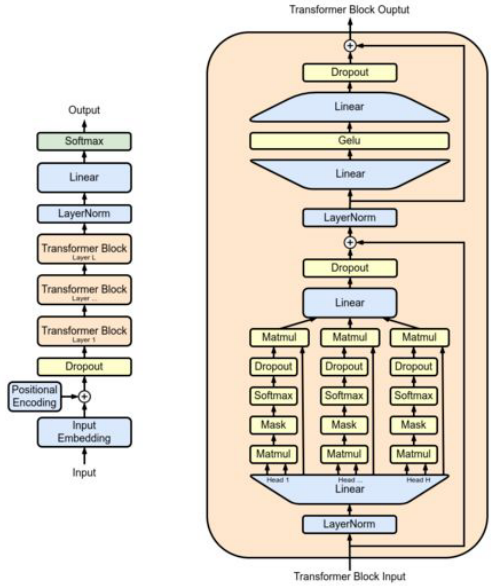
CNN

- Implement convolutional layers of LeNet-5 CNN
  - Inference (forward) pass only
- Individual project



GPT-2

- Implement layers in GPT-2 model
  - Inference (forward) pass only
- Team project (3-4 students per team)



Tentative Timeline

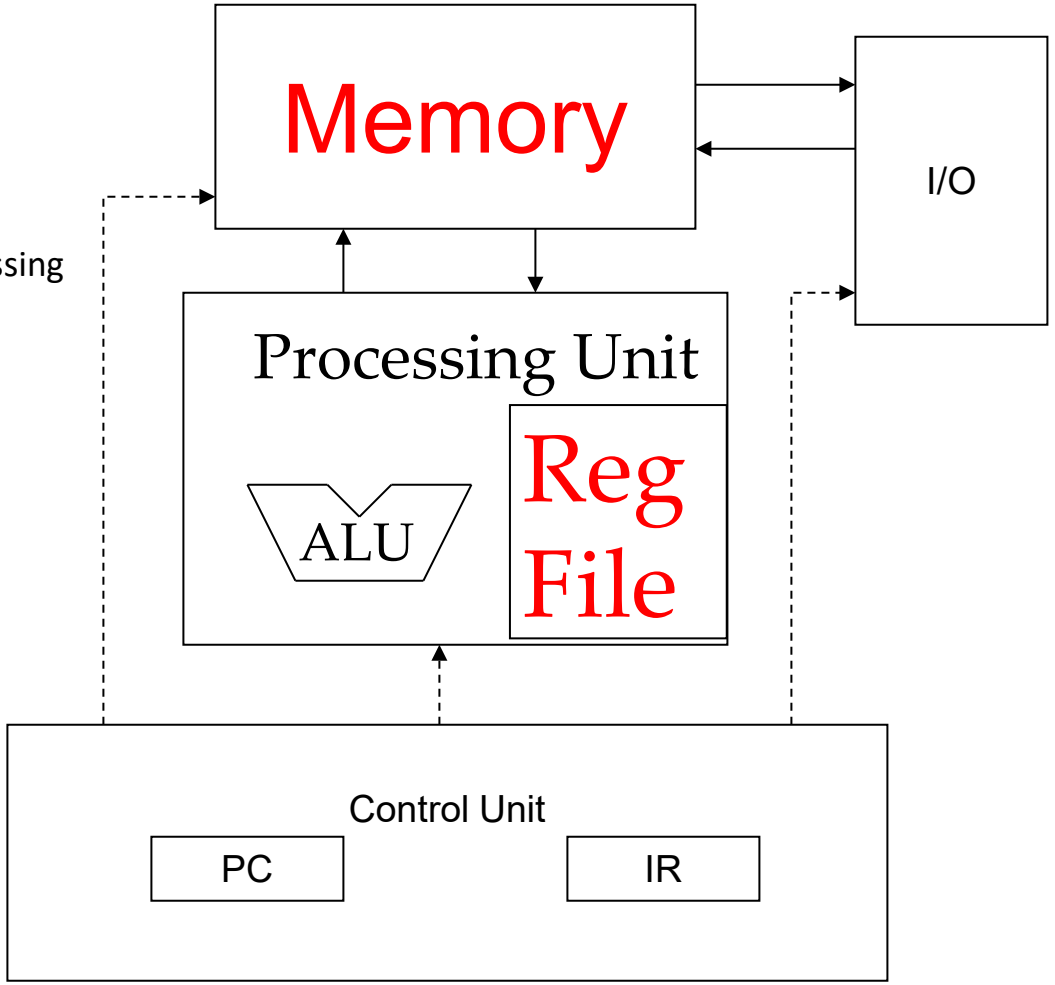
Milestone	Tentative Deadline
Milestone 1	Week 7-8
Milestone 2	Week 10-11
Milestone 3	Week 15-16

Today's Objectives

- To learn the basic features of the memories accessible by CUDA threads
- To prepare for Lab 2 - basic matrix multiplication
- To learn to evaluate the performance implications of global memory accesses
- To learn about GPU performance metrics

The Von-Neumann Model

- A computer based on the **Von Neumann architecture** consists of four main components:
  - **Processing Unit (PU)**
    - Performs all arithmetic and logical operations
    - Includes the **Register File**, where data is temporarily stored for processing
  - **Memory**
    - Stores both **program instructions** and **data**
  - **Input/Output (I/O) Subsystem**
    - Handles communication between the computer and the external environment
  - **Control Unit (CU)**
    - Directs the execution of instructions by coordinating all components
- All operations are performed on data stored in **registers** within the Processing Unit. Before any calculation:
  - **Data** must be fetched from **Memory** into registers, and
  - **Instructions** must be loaded from **Memory** into the **Instruction Register (IR)**



Instructions are Stored in Memory

- Every instruction needs to be fetched from memory, decoded, then executed

- Instruction processing breaks into steps:

Fetch | Decode | Execute | Memory

- Instructions come in three flavors:
  Operate, Data Transfer, and Control Flow

Example: Processing an ADD Instruction

- Example of an (LC-3) operate instruction:

  ADD R1, R2, R3

- meaning:
  - read R2 and R3
  - add them as unsigned/2's complement
  - write sum to R1

- Instruction processing for an operate instruction:

Fetch | Decode | Execute | Memory

Example: Processing a LOAD Instruction

- Example of an (LC-3) data transfer instruction:

  LDR R4, R6, #3 ; a load

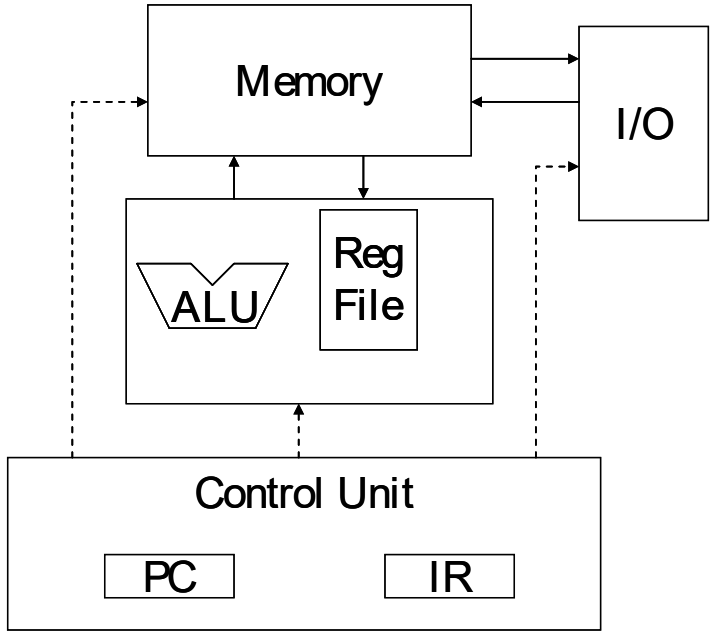
- meaning:
  - read R6
  - add the number 3 to it
  - load the contents of memory at the resulting address
  - write the bits to R4

- Instruction processing for a load instruction:

Fetch | Decode | Execute | Memory

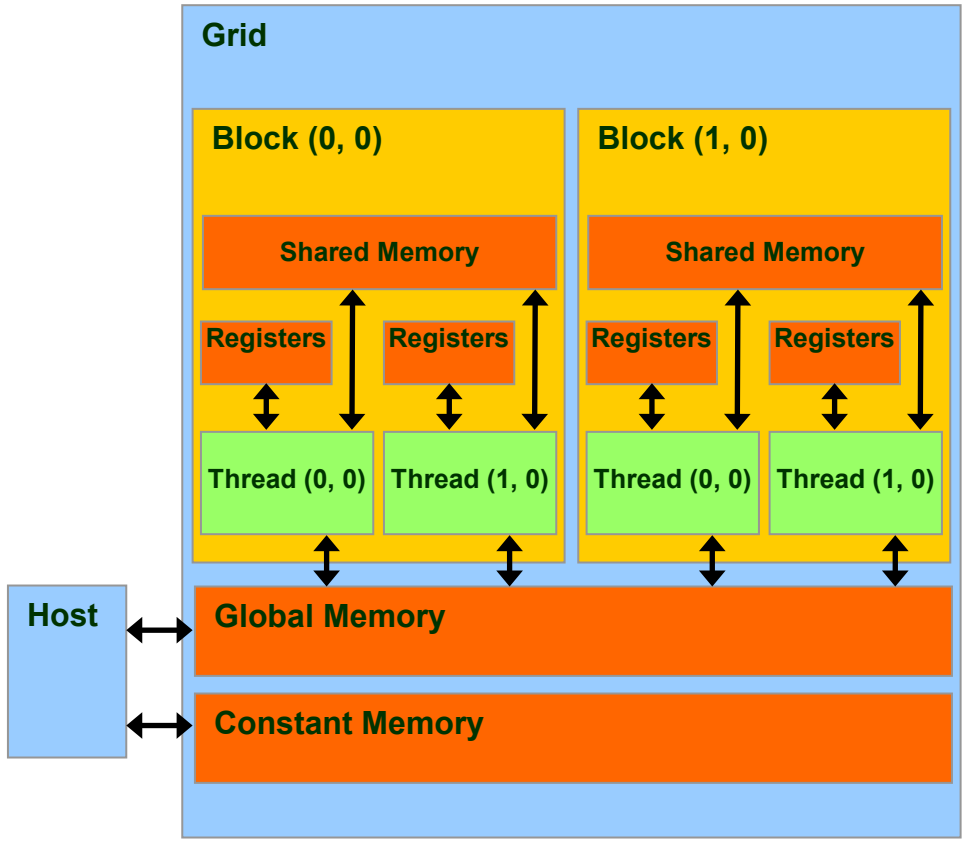
Registers vs Memory

- Registers
  - Fast: 1 cycle; no memory access required
  - Few: hundreds for CPU, O(10k) for GPU SM
- Memory
  - Slow: hundreds of cycles
  - Huge: GB or more

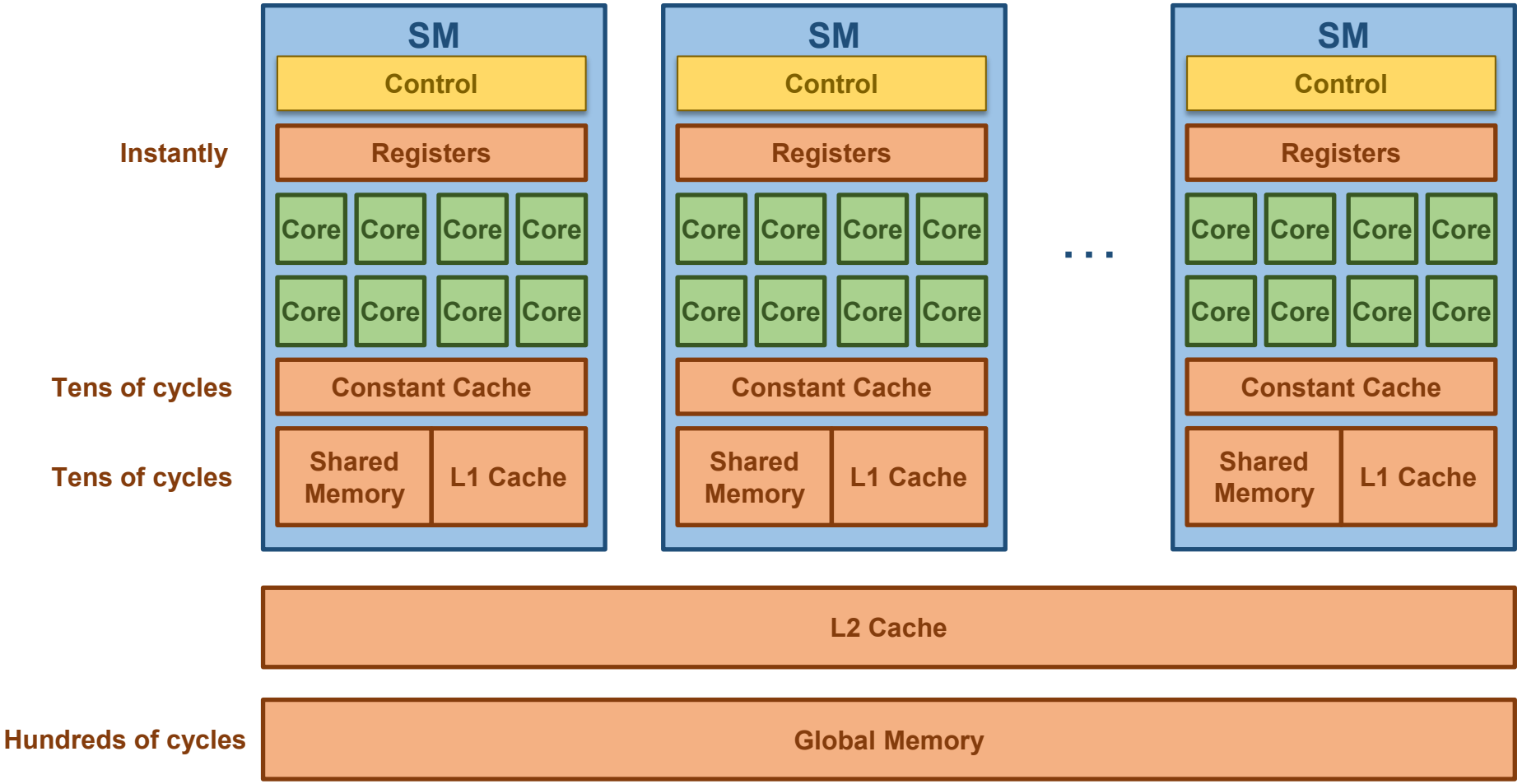


Programmer's View of CUDA Memories

- Each thread can:
  - read/write per-thread **registers (~1 cycle)**
  - read/write per-block **shared memory (~5 cycles)**
  - read/write per-grid **global memory (~500 cycles)**
  - read/only per-grid **constant memory (~5 cycles with caching)**



Memory in the GPU Architecture



CUDA Type Qualifies

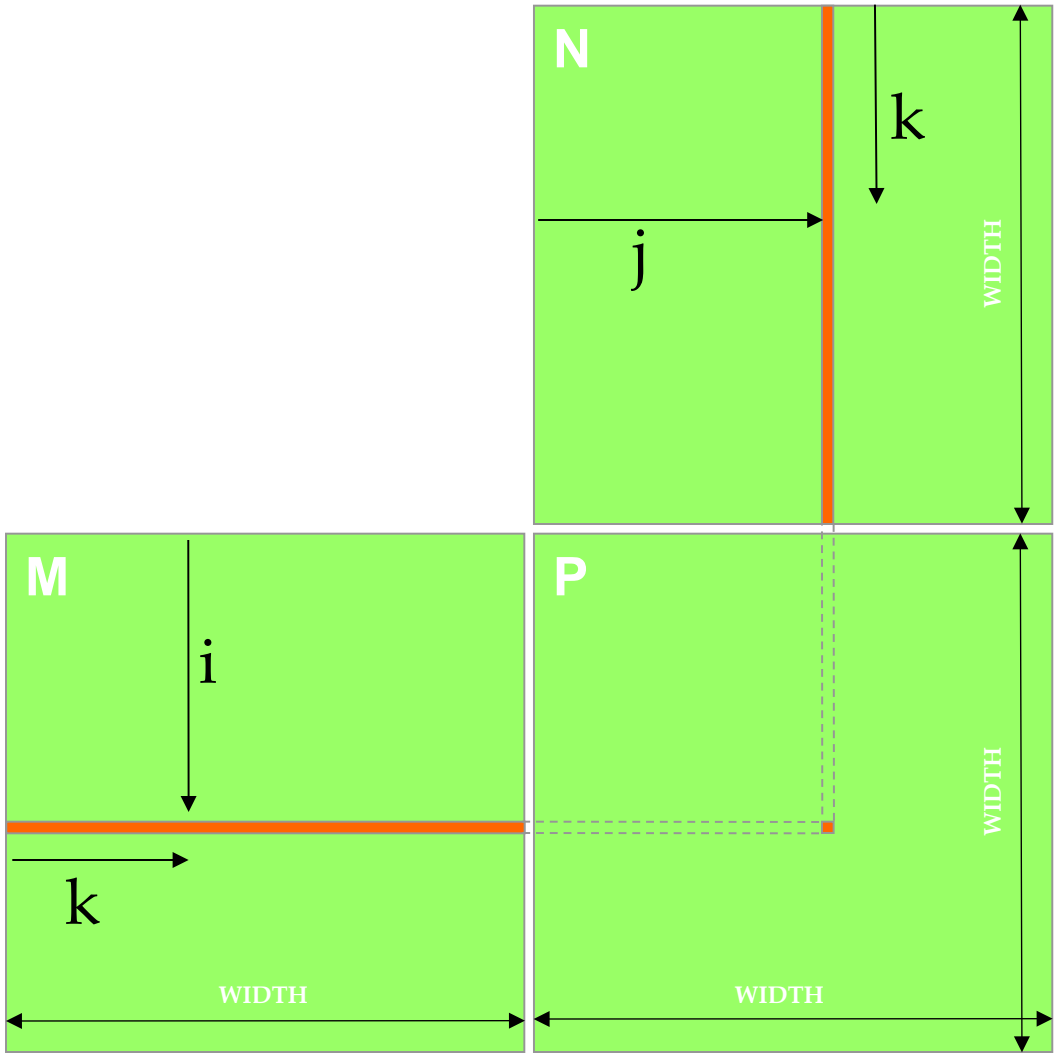
Variable declaration		Memory	Scope	Lifetime
`__device__`	`int globalVar;`	global	grid	application
`__device__ __constant__`	`int constantVar;`	constant	grid	application
`__device__ __shared__`	`int sharedVar;`	shared	block	block
	`int localVar;`	register	thread	thread
	`int localArr[N];`	global	thread	thread

- `__device__`
  - optional with `__shared__` or `__constant__`
  - not allowed by itself within functions
- Automatic variables with no qualifiers
  - in registers for primitive types and structures
  - in global memory for per-thread arrays

Matrix Multiplication

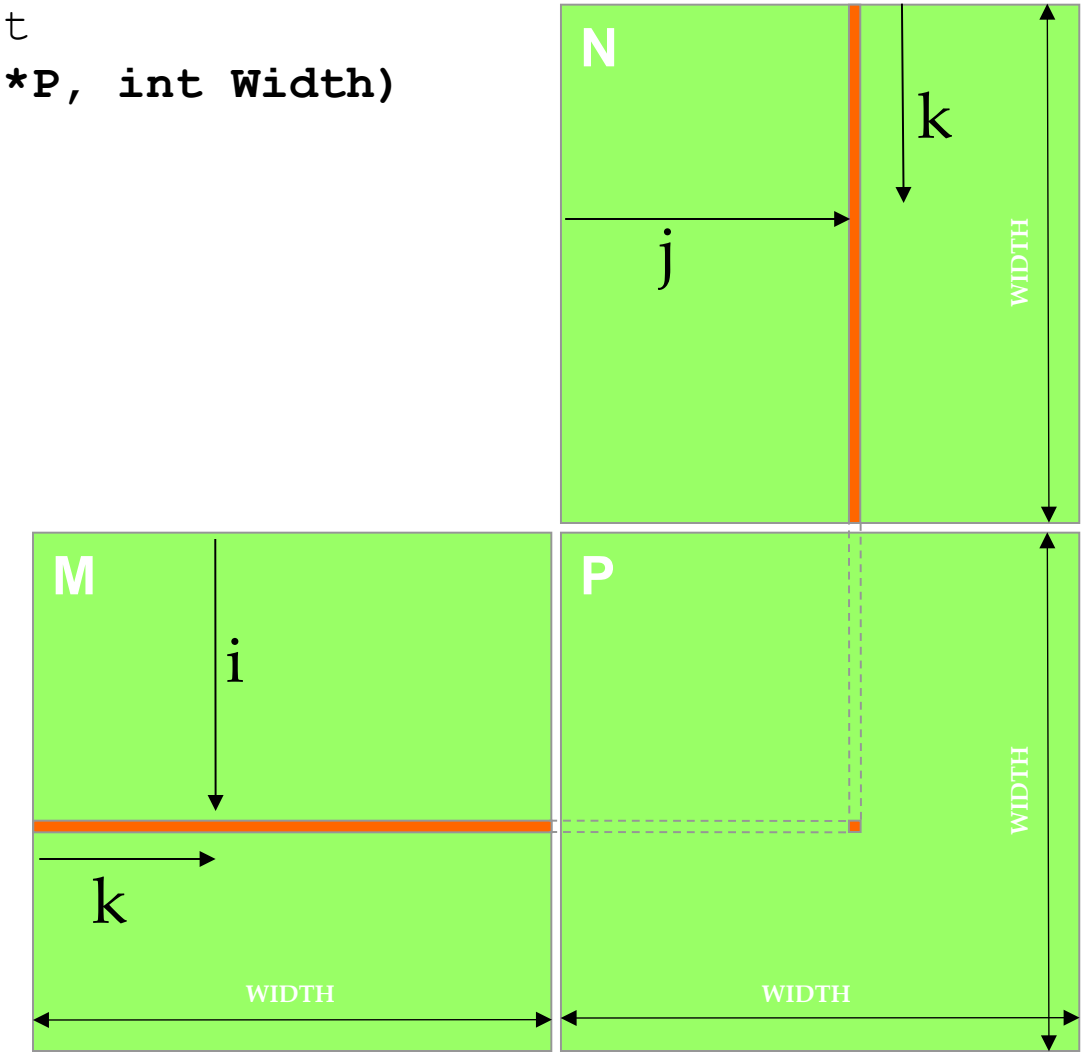
- $P_{ij} = \sum_{k=1}^{Width} M_{ik} N_{kj}$

- Graphically, imagine
  - taking each element in a row of M,
  - multiplying it by the corresponding element in a column of N, and
  - summing up the products
- Do that for every row and every column to produce P



Matrix Multiplication: A Simple Host Version in C

```
// Matrix multiplication on the (CPU) host
void MatrixMul(float *M, float *N, float *P, int Width)
{
   for (int i = 0; i < Width; ++i)
      for (int j = 0; j < Width; ++j)
      {
         float sum = 0;
         for (int k = 0; k < Width; ++k)
         {
            float a = M[i * Width + k];
            float b = N[k * Width + j];
            sum += a * b;
         }
         P[i * Width + j] = sum;
      }
}
```



Parallelize Elements of P

- What can we parallelize?
  - start with the **two outer loops**
  - parallelize **computation of elements of P**

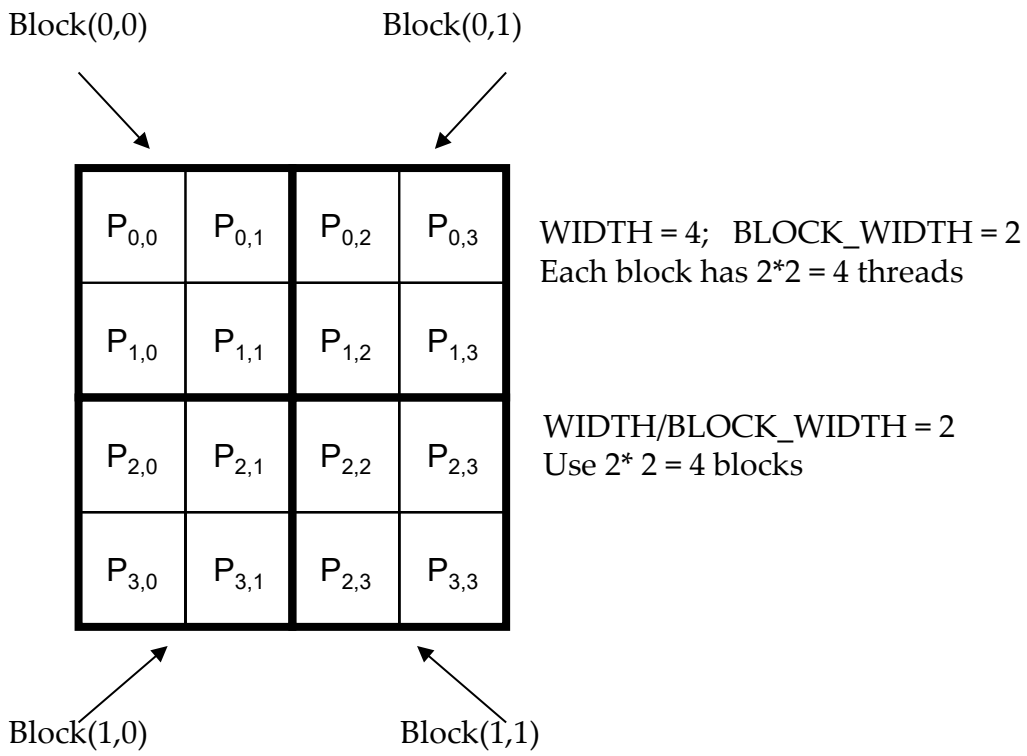
- What about the inner loop?
  - Technically, floating-point is NOT associative
  - The **parallel sum** is called a **reduction**—we'll come back to it in a few weeks
  - For now, **use a single thread for each P_{ij}**

Compute Using 2D Blocks in a 2D Grid

- **P** is 2D, so organize threads in 2D as well:
  - Split the output **P** into square **tiles**
    - **of size TILE_WIDTH × TILE_WIDTH**
    - (a preprocessor constant)
  - **Each thread block produces one tile** of TILE_WIDTH^2 elements
  - Create **[ceil (Width / TILE_WIDTH)]^2** thread **blocks** to cover the output matrix

Kernel Function - A Small Example

- Have each 2D thread block to compute a $(BLOCK_WIDTH)^2$ sub-matrix of the result matrix
  - Each block has $(BLOCK_WIDTH)^2$ threads
- Generate a 2D Grid of $(WIDTH/BLOCK_WIDTH)^2$ blocks
- This concept is called **tiling**
  - Each block represents a **tile**



Example: Width 8, TILE_WIDTH 2

- Each block has 2×2 = 4 threads
- WIDTH/TILE_WIDTH = 4
- Use 4×4 = 16 blocks

$P_{0,0}$	$P_{0,1}$	$P_{0,2}$	$P_{0,3}$	$P_{0,4}$	$P_{0,5}$	$P_{0,6}$	$P_{0,7}$
$P_{1,0}$	$P_{1,1}$	$P_{1,2}$	$P_{1,3}$	$P_{1,4}$	$P_{1,5}$	$P_{1,6}$	$P_{1,7}$
$P_{2,0}$	$P_{2,1}$	$P_{2,2}$	$P_{2,3}$	$P_{2,4}$	$P_{2,5}$	$P_{2,6}$	$P_{2,7}$
$P_{3,0}$	$P_{3,1}$	$P_{3,2}$	$P_{3,3}$	$P_{3,4}$	$P_{3,5}$	$P_{3,6}$	$P_{3,7}$
$P_{4,0}$	$P_{4,1}$	$P_{4,2}$	$P_{4,3}$	$P_{4,4}$	$P_{4,5}$	$P_{4,6}$	$P_{4,7}$
$P_{5,0}$	$P_{5,1}$	$P_{5,2}$	$P_{5,3}$	$P_{5,4}$	$P_{5,5}$	$P_{5,6}$	$P_{5,7}$
$P_{6,0}$	$P_{6,1}$	$P_{6,2}$	$P_{6,3}$	$P_{6,4}$	$P_{6,5}$	$P_{6,6}$	$P_{6,7}$
$P_{7,0}$	$P_{7,1}$	$P_{7,2}$	$P_{7,3}$	$P_{7,4}$	$P_{7,5}$	$P_{7,6}$	$P_{7,7}$

Example: Larger Tiles (Width 8, TILE_WIDTH 4)

- Each block has 4×4 = 16 threads
- WIDTH/TILE_WIDTH = 2
- Use 2×2 = 4 blocks

$P_{0,0}$	$P_{0,1}$	$P_{0,2}$	$P_{0,3}$	$P_{0,4}$	$P_{0,5}$	$P_{0,6}$	$P_{0,7}$
$P_{1,0}$	$P_{1,1}$	$P_{1,2}$	$P_{1,3}$	$P_{1,4}$	$P_{1,5}$	$P_{1,6}$	$P_{1,7}$
$P_{2,0}$	$P_{2,1}$	$P_{2,2}$	$P_{2,3}$	$P_{2,4}$	$P_{2,5}$	$P_{2,6}$	$P_{2,7}$
$P_{3,0}$	$P_{3,1}$	$P_{3,2}$	$P_{3,3}$	$P_{3,4}$	$P_{3,5}$	$P_{3,6}$	$P_{3,7}$
$P_{4,0}$	$P_{4,1}$	$P_{4,2}$	$P_{4,3}$	$P_{4,4}$	$P_{4,5}$	$P_{4,6}$	$P_{4,7}$
$P_{5,0}$	$P_{5,1}$	$P_{5,2}$	$P_{5,3}$	$P_{5,4}$	$P_{5,5}$	$P_{5,6}$	$P_{5,7}$
$P_{6,0}$	$P_{6,1}$	$P_{6,2}$	$P_{6,3}$	$P_{6,4}$	$P_{6,5}$	$P_{6,6}$	$P_{6,7}$
$P_{7,0}$	$P_{7,1}$	$P_{7,2}$	$P_{7,3}$	$P_{7,4}$	$P_{7,5}$	$P_{7,6}$	$P_{7,7}$

Kernel Invocation (Host-side Code)

```
// TILE_WIDTH is a #define constant
dim3 dimGrid(ceil((1.0*Width)/TILE_WIDTH),
        ceil((1.0*Width)/TILE_WIDTH), 1);

dim3 dimBlock(TILE_WIDTH, TILE_WIDTH, 1);


// Launch the device computation threads!
MatrixMulKernel<<<dimGrid, dimBlock>>>(Md, Nd, Pd, Width);
```

Kernel Function

```
// Matrix multiplication kernel – per thread code
__global__
void MatrixMulKernel(float* d_M, float* d_N, float* d_P, int Width)
{
    // Calculate the row index of the d_P element and d_M
    int Row = blockIdx.y * blockDim.y + threadIdx.y;

    // Calculate the column idenx of d_P and d_N
    int Col = blockIdx.x * blockDim.x + threadIdx.x;

    // Pvalue is used to store the element of the matrix
    // that is computed by the thread
    float Pvalue = 0;
    ...
    ...
    d_P[  ] = Pvalue;
}
```

Work for Block (0,0) with TILE_WIDTH 2

blockDim.x

blockDim.y

Col = 0 * 2 + threadIdx.x

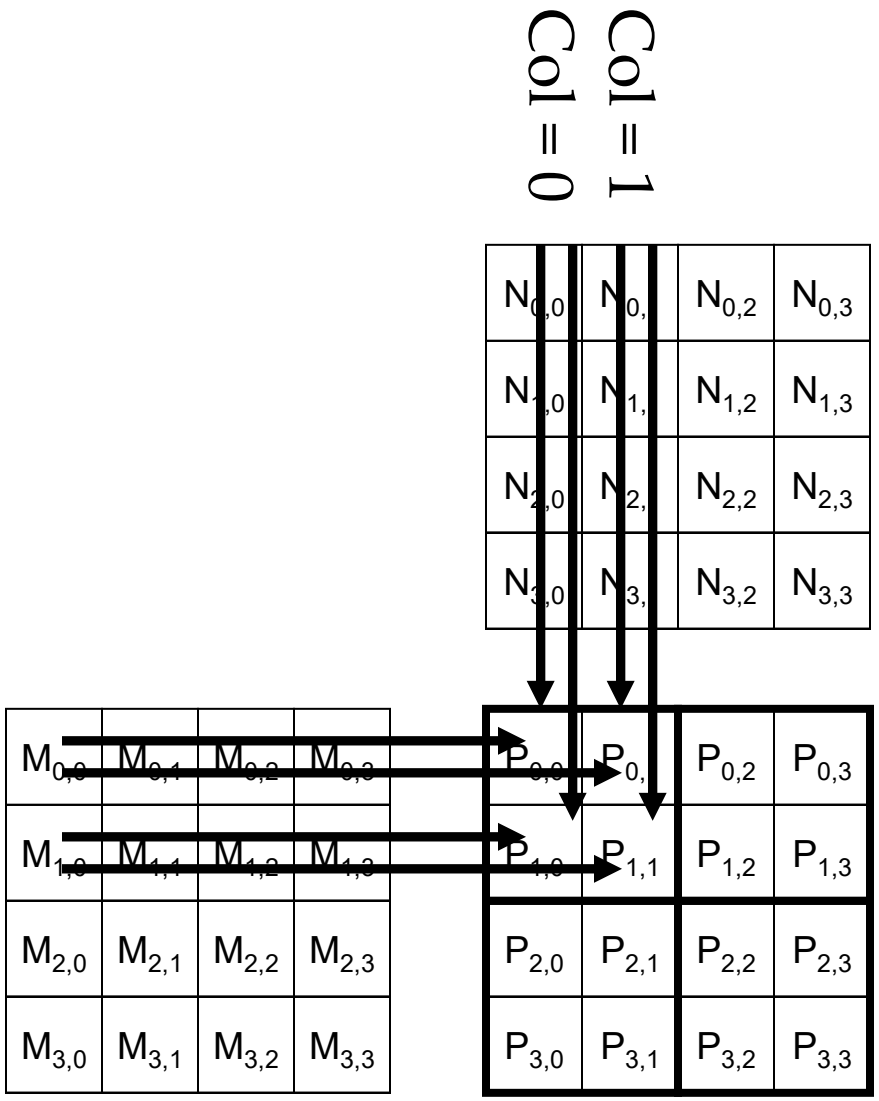
Row = 0 * 2 + threadIdx.y

blockIdx.x

blockIdx.y

Row = 0

Row = 1



Work for Block (0,1)

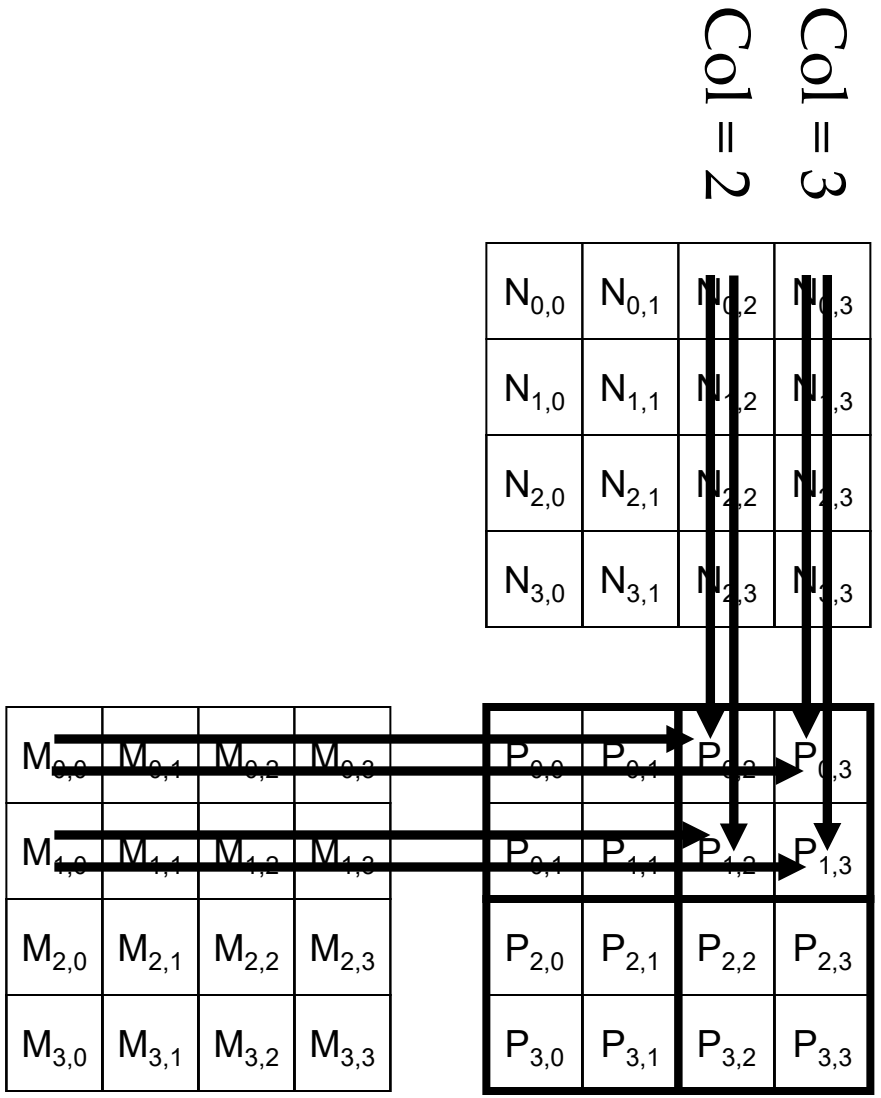
Col = 1 * 2 + threadIdx.x
Row = 0 * 2 + threadIdx.y

blockIdx.x

blockIdx.y

Row = 0

Row = 1



A Simple Matrix Multiplication Kernel

```
__global__
void MatrixMulKernel(float* d_M, float* d_N, float* d_P, int Width)
{
   // Calculate the row index of the d_P element and d_M
   int Row = blockIdx.y*blockDim.y+threadIdx.y;
   // Calculate the column idenx of d_P and d_N
   int Col = blockIdx.x*blockDim.x+threadIdx.x;

   if ((Row < Width) && (Col < Width)) {
      float Pvalue = 0;
      // each thread computes one element of the block sub-matrix
      for (int k = 0; k < Width; ++k)
          Pvalue += d_M[Row*Width+k] * d_N[k*Width+Col];

      d_P[Row*Width+Col] = Pvalue;
   }
}
```

Memory Bandwidth is Overloaded!

- That's a **simple implementation**:
  - GPU kernel is the **CPU code** with the **outer loops replaced with** per-thread **index calculations**!
- Unfortunately, performance is quite bad.

- Why?

- With the given approach,
  - global **memory bandwidth can't** supply enough data to **keep the SMs busy**!

Where Do We Access Global Memory?

```
__global__
void MatrixMulKernel(float* d_M, float* d_N, float* d_P, int Width)
{
   // Calculate the row index of the d_P element and d_M
   int Row = blockIdx.y*blockDim.y+threadIdx.y;
   // Calculate the column idenx of d_P and d_N
   int Col = blockIdx.x*blockDim.x+threadIdx.x;

   if ((Row < Width) && (Col < Width)) {
      float Pvalue = 0;
      // each thread computes one element of the block sub-matrix
      for (int k = 0; k < Width; ++k)
          Pvalue += d_M[Row*Width+k] * d_N[k*Width+Col];

      d_P[Row*Width+Col] = Pvalue;
   }
}
```

Each Thread Requires 4B of Data per FLOP

- Every threads access global memory
  - for elements of **M** and **N**:
  - **4B each**, or **8B per pair**
  - (And once TOTAL to **P** per thread—ignore it.)
- With each pair of elements,
  - a thread does a single multiply-add,
  - **2 FLOP**—floating-point operations
- So for every FLOP,
  - **a thread needs** 4B from memory:
  - **4B / FLOP**

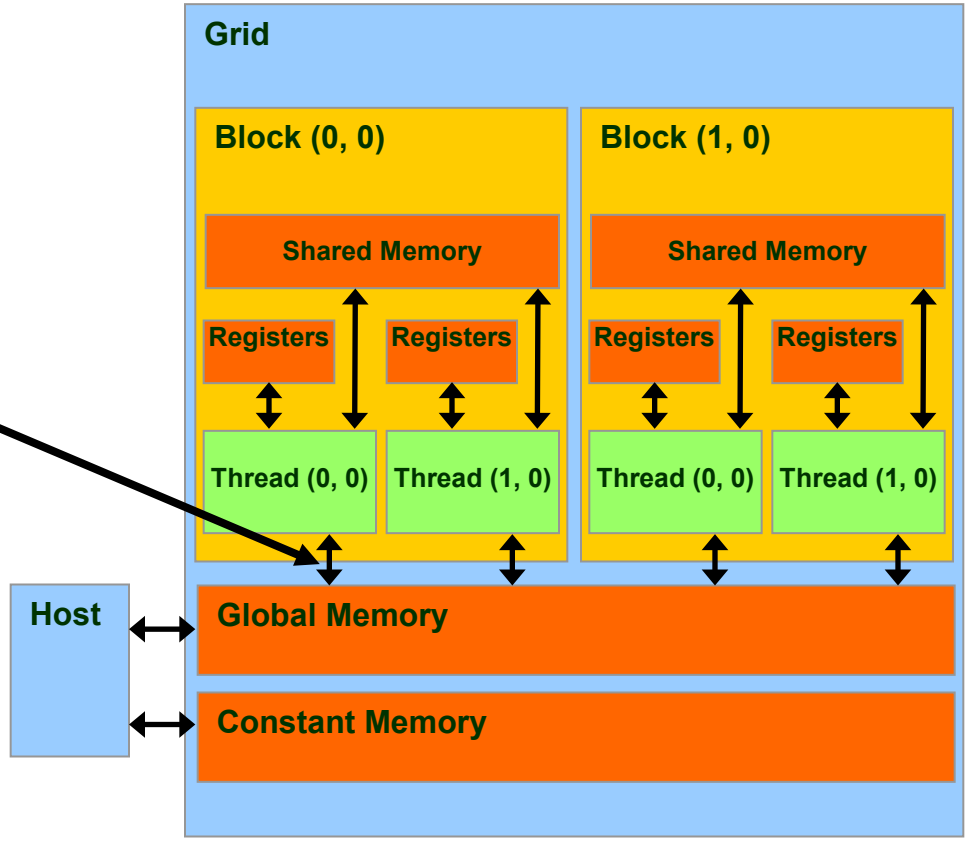
150 GB/s Bandwidth Implies 37.5 GFLOPs

- One generation of GPUs:
  - **1,000 GFLOP/s** of compute power, and
  - **150 GB/s** of memory bandwidth

- Dividing bandwidth by memory requirements:

$$\frac{150\ GB/s}{4\ B/FLOP} = 37.5\ \text{GFLOP/s}$$

which **limits computation**!



What to Do? Reuse Memory Accesses!

But **37.5 GFLOPs is a limit**

In an **actual execution**,

- memory is not busy all the time, and
- the code **runs at** about **25 GFLOPs**

To get closer to 1,000 GFLOPs

- we **need to** drastically **cut down**
- **accesses to global memory** (next lecture)

Performance Metrics

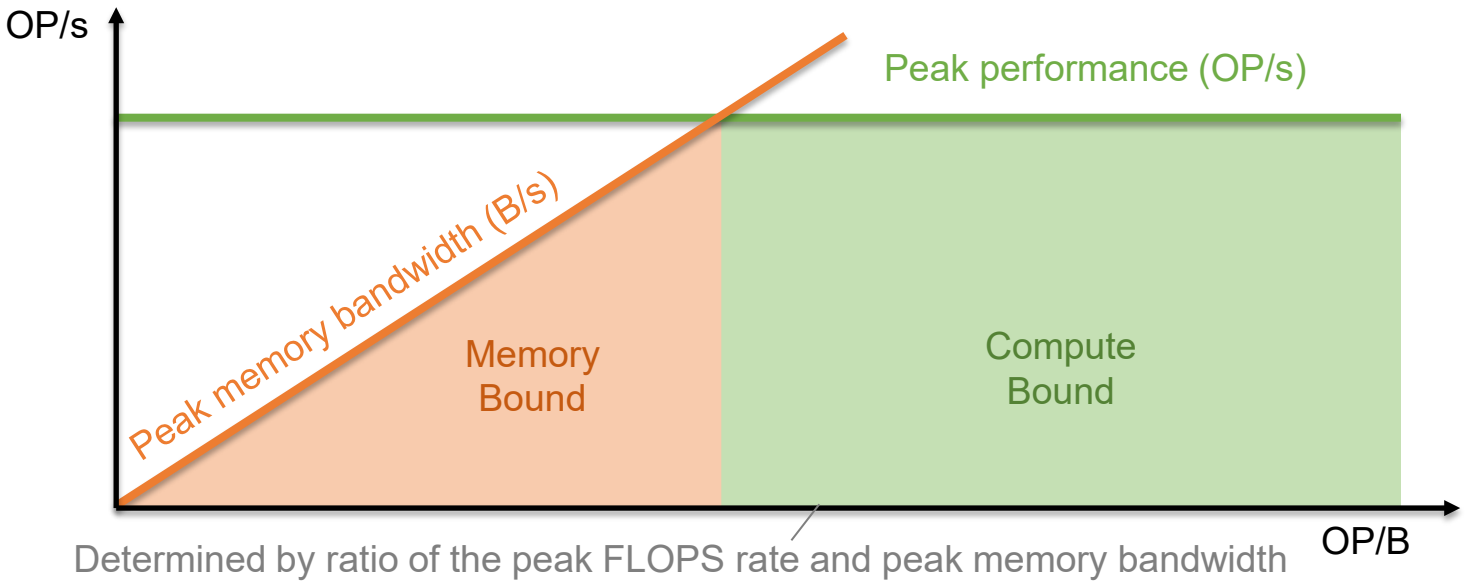
- Processor designers inform users about a processor's performance through various metrics, most notably:
  - **FLOPS Rate**: floating point operations per second
    - How much computation a processor's cores can do per unit time
  - **Memory Bandwidth**: bytes per second
    - How much data the memory can supply to the cores per unit time

GPU Name	K40	M40	P100	V100	A100	H100
Microarchitecture	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
Peak FLOPS Rate (GFLOPS)	5,040	6,844	9,340	14,028	19,500	66,900
Peak Memory Bandwidth (GB/s)	288	288	732	900	1,555	3,352

- These metrics are *not a guarantee*
  - They are a *limit* of the maximum possible performance

Performance Bound and the Roofline Model

- A kernel can be:
  - **Compute-bound**: performance limited by the FLOPS rate
    - The processor's cores are fully utilized (always have work to do)
  - **Memory-bound**: performance limited by the memory bandwidth
    - The processor's cores are frequently idle because memory cannot supply data fast enough
- The **roofline model** helps visualize a kernel's performance bound based on the ratio of operations it performs and bytes it accesses from memory



Ratio of Peak FLOPS Rate to Peak Bandwidth

- The OP/B ratio that separates compute-bound and memory-bound kernels is hardware dependent

GPU Name	K40	M40	P100	V100	A100	H100
Microarchitecture	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
Peak FLOPS Rate (GFLOPS)	5,040	6,844	9,340	14,028	19,500	66,900
Peak Memory Bandwidth (GB/s)	288	288	732	900	1,555	3,352
Ratio (OP/B)	17.5	23.8	12.8	15.6	12.5	20.0

- Finding a kernel's OP/B allows us to determine if a kernel is memory-bound or compute-bound on a specific hardware
- Knowing the kernel's bound allows us to determine the best possible performance achievable by the kernel (sometimes called **speed of light**)

Example 1: Vector Addition

```
z[i] = x[i] + y[i];
```

- Vector addition performs 0.083 OP/B
  - 1 FP add operation for every 2 FP values (8B) loaded and 1 FP value (4B) stored

- Vector addition is memory-bound
  - Performance is limited by the peak memory bandwidth
  - To add 1 billion elements on an H100 GPU, the speed of light performance is:

$$\frac{1\ GOP}{(0.083\ OP/B) \cdot (3.35\ TB/s)} = 3.60\ ms$$

- Comparing our kernel's performance to the speed of light performance allows us to assess how well-optimized our kernel is

Example 2: Matrix-Matrix Multiplication

- For matrix multiplication of N×N square matrices
  - Data loaded/stored: (3 matrices)×(N^2 values)×(4 B) = $12N^2$ B
  - Operations: (N^2 dot products)×(N adds + N muls each) = $2N^3$ OP
  - Ratio: 0.167N OP/B
    - For large N, matrix multiplication should be highly compute-bound
- What about our previous implementation?

```
for(unsigned int i = 0; i < N; ++i) {
    sum += A[row*N + i] * B[i*N + col];
}
```

  - 1 FP add and 1 FP mul for every 2 FP values (8B) loaded
    - Ignoring final store which is negligible for large N
  - Ratio: 0.25 OP/B
  - The achieved ratio is low because the same values are loaded multiple times
    - Better in practice because some values are reused from the cache
    - Can do even better if we can control how the values are reused

Any More Questions?

Things to Read:

Textbook chapter 3

CUDA PG: Programming Model

Things to Do:

Get Delta account

Setup GitHub

Submit Labs 0, 1, and 2

Problem Solving

- Q: Consider a 2D input matrix of 256 rows and 4 columns. We use 16 x 16 block of threads to perform operations on the input matrix so that each thread processes exactly one input element. How many blocks need to be launched?

- A:
  - 256 rows * 4 columns = 1024 elements to compute
  - 16 * 16 =256 threads per block
  - 1024 / 256 = 4 blocks of threads

Problem Solving

- Q: Suppose your kernel code requires certain threads to read data items written by other threads in the same thread block. Which type of memory will be most suitable (fastest accesses) for this purpose?
  - Register
  - Shared memory
  - Global memory

- A:
  - Shared memory

Problem Solving

- Q: What are the possible values of *dst after this kernel execution?

```
__global__ void kernel(char *dst) {
    dst[0] = blockIdx.x;
}

// dst is a pointer to an array of one char allocated on
// the device and initialized to value of 3, e.g., dst[0]=3
kernel<<<2,1>>>(dst);
```

- A:
  - Either 0 or 1